CSC2161
C++ OOP

Dieudonne

What are functions of C++?

The major purpose of C++ programming is to introduce the concept of object orientation to the C programming language.

Object Oriented Programming is a paradigm that provides many concepts such as **inheritance, data binding, polymorphism etc.**

The programming paradigm where everything is represented as an object is known as truly object-oriented programming language.

.



Class

Collection of objects is called class. It is a logical entity.

A Class in C++ is the foundational element that leads to Object-Oriented programming. A class instance must be created in order to access and use the user-defined data type's data members and member functions. An object's class acts as its blueprint. Take the class of cars as an example. Even if different names and brands may be used for different cars, all of them will have some characteristics in common, such as four wheels, a speed limit, a range of miles, etc. In this case, the class of car is represented by the wheels, the speed limitations, and the mileage.

Object

Any entity that has state and behavior is known as an object. For example: chair, pen, table, keyboard, bike etc. It can be physical and logical.

Inheritance

When one object acquires all the properties and behaviours of parent object i.e. known as inheritance. It provides code reusability. It is used to achieve runtime polymorphism.

Sub class - Subclass or Derived Class refers to a class that receives properties from another class.

Super class - The term "Base Class" or "Super Class" refers to the class from which a subclass inherits its properties.

Reusability - As a result, when we wish to create a new class, but an existing class already contains some of the code we need, we can generate our new class from the old class thanks to inheritance. This allows us to utilize the fields and methods of the pre-existing class.

Polymorphism

When one task is performed by different ways i.e. known as polymorphism. For example: to convince the customer differently, to draw something e.g. shape or rectangle etc.

Different situations may cause an operation to behave differently. The type of data utilized in the operation determines the behavior.

Abstraction

Hiding internal details and showing functionality is known as abstraction. Data abstraction is the process of exposing to the outside world only the information that is absolutely necessary while concealing implementation or background information.For example: phone call, we don't know the internal processing.

In C++, we use abstract class and interface to achieve abstraction.

Encapsulation

Binding (or wrapping) code and data together into a single unit is known as encapsulation. For example: capsule, it is wrapped with different medicines.

Encapsulation is typically understood as the grouping of related pieces of information and data into a single entity. Encapsulation is the process of tying together data and the functions that work with it in object-oriented programming. Take a look at a practical illustration of encapsulation: at a company, there are various divisions, including the sales division, the finance division, and the accounts division. All financial transactions are handled by the finance sector, which also maintains records of all financial data. In a similar vein, the sales section is in charge of all tasks relating to sales and maintains a record of each sale.

Now, a scenario could occur when, for some reason, a financial official requires all the information on sales for a specific month. Under the umbrella term "sales section," all of the employees who can influence the sales section's data are grouped together. Data abstraction or concealing is another side effect of encapsulation. In the same way that encapsulation hides the data. In the aforementioned example, any other area cannot access any of the data from any of the sections, such as sales, finance, or accounts.

Advantages of OOPs over Procedure-oriented programming language

OOPs makes development and maintenance easier where as in Procedure-oriented programming language it is not easy to manage if code grows as project size grows.

OOPs provide data hiding whereas in Procedure-oriented programming language a global data can be accessed from anywhere.

OOPs provide ability to simulate real-world event much more effectively. We can provide the solution of real word problem if we are using the Object-Oriented Programming language.

Overloading

Polymorphism also has a subset known as overloading.

An existing operator or function is said to be overloaded when it is forced to operate on a new data type.

C++ Object and Class

Since C++ is an object-oriented language, program is designed using objects and classes in C++.

Creating C++ Classes and Objects

In C++, class is a group of similar objects. It is a template from which objects are created. It can have fields, methods, constructors etc.

Let's see an example of C++ class that has three fields only.

```
class Student
 {
    public:
    int id;  //field or data member
    String name;//field or data member
 }
```

```
#include <iostream>
using namespace std;
class Student {
  public:
    int id;//data member (also instance variable)
    string name;//data member(also instance variable)
};
int main() {
   Student s1; //creating an object of Student
   s1.id = 201;
   s1.name = "Sonoo Jaiswal";
   cout<<s1.id<<endl;
   cout<<s1.name<<endl;
   return 0;
}
```

Initialize and Display data through method

```
#include <iostream>
using namespace std;
class Student {
  public:
     int id;//data member (also instance variable)
     string name;//data member(also instance variable)
     void insert(int i, string n)
     {
        id = i;
        name = n;
     }
     void display()
     {
        cout<<id<<"  "<<name<<endl;
     }
};
int main(void) {
    Student s1; //creating an object of Student
    Student s2; //creating an object of Student
    s1.insert(201, "Sonoo");
    s2.insert(202, "Nakul");
    s1.display();
    s2.display();
    return 0;
}
```

C++ Constructor

In C++, constructor is a special method which is invoked automatically at the time of object creation. It is used to initialize the data members of new object generally. The constructor in C++ has the same name as class or structure.

In brief, A particular procedure called a constructor is called automatically when an object is created in C++. In general, it is employed to create the data members of new things. In C++, the class or structure name also serves as the constructor name. When an object is completed, the constructor is called. Because it creates the values or gives data for the thing, it is known as a constructor.

The Constructors prototype looks like this:

<**class**-name> (list-of-parameters);

The following syntax is used to define the class's constructor:

<**class**-name> (list-of-parameters) { // constructor definition }

The following syntax is used to define a constructor outside of a class:

<**class**-name>: :<**class**-name> (list-of-parameters){ // constructor definition}

Constructors lack a return type since they don't have a return value.

There can be two types of constructors in C++.

Default constructor

Parameterized constructor

C++ Default Constructor

A constructor which has no argument is known as default constructor. It is invoked at the time of creating object. Let's see the simple example of C++ default Constructor.

```
#include <iostream>
using namespace std;
class Employee
 {
   public:
       Employee()
       {
          cout<<"Default Constructor Invoked"<<endl;
       }
};
int main(void)
{
    Employee e1; //creating an object of Employee
    Employee e2;
    return 0;
}
```

C++ Parameterized Constructor

Constructor which has parameters is called parameterized constructor. It is used to provide different values to distinct objects. Let's see the simple example of C++ Parameterized Constructor.

```
// part 1 of 2
#include <iostream>
using namespace std;
class Employee {
  public:
    int id;//data member (also instance variable)
    string name;//data member(also instance variable)
    float salary;
    Employee(int i, string n, float s)
    {
        id = i;
        name = n;
        salary = s;
    }
```

```
//Part 2 of 2
void display()
    {
        cout<<id<<"  "<<name<<"  "<<salary<<endl;
    }
};

int main(void) {
    Employee e1 =Employee(101, "Sonoo", 890000); // creating an object of Employee
    Employee e2=Employee(102, "Nakul", 59000);
    e1.display();
    e2.display();
    return 0;
}
```

What is a destructor in C++?

A destructor works opposite to constructor; it destructs the objects of classes. It can be defined only once in a class. Like constructors, it is invoked automatically.

A destructor is defined like constructor. It must have same name as class. But it is prefixed with a tilde sign (~).

C++ Constructor and Destructor Example

```
include <iostream>
using namespace std;
class Employee
 {
   public:
       Employee()
       {
          cout<<"Constructor Invoked"<<endl;
       }
       ~Employee()
       {
          cout<<"Destructor Invoked"<<endl;
       }
};
```

```
int main(void)
{
   Employee e1; //creating an object of Employee
   Employee e2; //creating an object of Employee
   return 0;
}
```

Output

```
Constructor Invoked
Constructor Invoked
Destructor Invoked
Destructor Invoked
```

C++ this Pointer

In C++ programming, **this** is a keyword that refers to the current instance of the class. There can be 3 main usage of this keyword in C++.

It can be used **to pass current object as a parameter to another method.**

It can be used **to refer current class instance variable.**

It can be used **to declare indexers.**

```
#include <iostream>
using namespace std;
class Employee {
  public:
     int id; //data member (also instance variable)
     string name; //data member(also instance variable)
     float salary;
     Employee(int id, string name, float salary)
     {
         this->id = id;
        this->name = name;
        this->salary = salary;
     }
     void display()
     {
        cout<<id<<"  "<<name<<"  "<<salary<<endl;
     }
};
int main(void) {
   Employee e1 =Employee(101, "Sonoo", 890000); //
creating an object of Employee
   Employee e2=Employee(102, "Nakul", 59000); //
creating an object of Employee
   e1.display();
   e2.display();
   return 0;
}
```

Output:

```
101  Sonoo  890000
102  Nakul  59000
```

Static

In C++, static is a keyword or modifier that belongs to the type not instance. So instance is not required to access the static members. In C++, static can be field, method, constructor, class, properties, operator and event.

Advantage of C++ static keyword

Memory efficient: Now we don't need to create instance for accessing the static members, so it saves memory. Moreover, it belongs to the type, so it will not get memory each time when instance is created.

Static Field

A field which is declared as static is called static field. Unlike instance field which gets memory each time whenever you create object, there is only one copy of static field created in the memory. It is shared to all the objects.

It is used to refer the common property of all objects such as rateOfInterest in case of Account, companyName in case of Employee etc.

```
#include <iostream>
using namespace std;
class Account {
  public:
     int accno; //data member (also instance variable)
     string name; //data member(also instance variable)
     static float rateOfInterest;
     Account(int accno, string name)
     {
         this->accno = accno;
        this->name = name;
     }
     void display()
     {
        cout<<accno<< "<<name<< " "<<rateOfInterest<<endl;
     }
};
```

```
float Account::rateOfInterest=6.5;
int main(void) {
   Account a1 =Account(201, "Sanjay"); //
creating an object of Employee
   Account a2=Account(202, "Nakul"); //
creating an object of Employee
   a1.display();
   a2.display();
   return 0;
}
```

Output:
201 Sanjay 6.5 202 Nakul 6.5

Access Specifiers

By now, you are quite familiar with the public keyword that appears in all of our class examples:

```
class MyClass {  // The class
  public:       // Access specifier
    // class members goes here
};
```

The public keyword is an access specifier. Access specifiers define how the members (attributes and methods) of a class can be accessed. In the example above, the members are public - which means that they can be accessed and modified from outside the code.

However, what if we want members to be private and hidden from the outside world?

In C++, there are three access specifiers:

public - members are accessible from outside the class

private - members cannot be accessed (or viewed) from outside the class

protected - members cannot be accessed from outside the class, however, they can be accessed in inherited classes. You will learn more about Inheritance later.

In the following example, we demonstrate the differences between `public` and `private` members:

```
class MyClass {
  public:    // Public access specifier
    int x;   // Public attribute
  private:   // Private access specifier
    int y;   // Private attribute
};

int main() {
  MyClass myObj;
  myObj.x = 25;  // Allowed (public)
  myObj.y = 50;  // Not allowed (private)
  return 0;
}
```

If you try to access a private member, an error occurs:

error: y is private

Note: By default, all members of a class are private if you don't specify an access specifier:

Example

```
class MyClass {
  int x;   // Private attribute
  int y;   // Private attribute
};
```

Struct

In C++, classes and structs are blueprints that are used to create the instance of a class. Structs are used for lightweight objects such as Rectangle, color, Point, etc.

Unlike class, structs in C++ are value type than reference type. It is useful if you have data that is not intended to be modified after creation of struct.

C++ Structure is a collection of different data types. It is similar to the class that holds different types of data.

The Syntax Of Structure

```
struct structure_name
{
    // member declarations.
}
```

In the above declaration, a structure is declared by preceding the **struct keyword** followed by the identifier(structure name). Inside the curly braces, we can declare the member variables of different types. **Consider the following situation:**

```
struct Student
{
    char name[20];
     int id;
     int age;
}
```

In the above case, Student is a structure contains three variables name, id, and age. When the structure is declared, no memory is allocated. When the variable of a structure is created, then the memory is allocated. Let's understand this scenario.

How to create the instance of Structure?

Structure variable can be defined as:

Student s;

How to access the variable of Structure:

The variable of the structure can be accessed by simply using the instance of the structure followed by the dot (.) operator and then the field of the structure.

For example:

s.id = 4;

In the above statement, we are accessing the id field of the structure Student by using the **dot(.)** operator and assigns the value 4 to the id field.

C++ Struct Example

Let's see a simple example of struct Rectangle which has two data members width and height.

```
#include <iostream>
using namespace std;
 struct Rectangle
{
   int width, height;

 };
int main(void) {
    struct Rectangle rec;
    rec.width=8;
    rec.height=5;
   cout<<"Area of Rectangle is: "<<(rec.width * rec.height)<<endl;
 return 0;
}
```

Structure v/s Class

Structure	Class
If access specifier is not declared explicitly, then by default access specifier will be public.	If access specifier is not declared explicitly, then by default access specifier will be private.

Syntax of Structure:
 	
struct structure_name
{
// body of the structure.
}
	Syntax of Class:
	
	class class_name
	{
	// body of the class.
	}

The instance of the structure is known as "Structure variable".	The instance of the class is known as "Object of the class".

Inheritence

In C++, inheritance is a process in which one object acquires all the properties and behaviors of its parent object automatically. In such way, you can reuse, extend or modify the attributes and behaviors which are defined in other class.

In C++, the class which inherits the members of another class is called derived class and the class whose members are inherited is called base class. The derived class is the specialized class for the base class.

Advantage of C++ Inheritance

Code reusability: Now you can reuse the members of your parent class. So, there is no need to define the member again. So less code is required in the class.

Types Of Inheritance

C++ supports five types of inheritance:

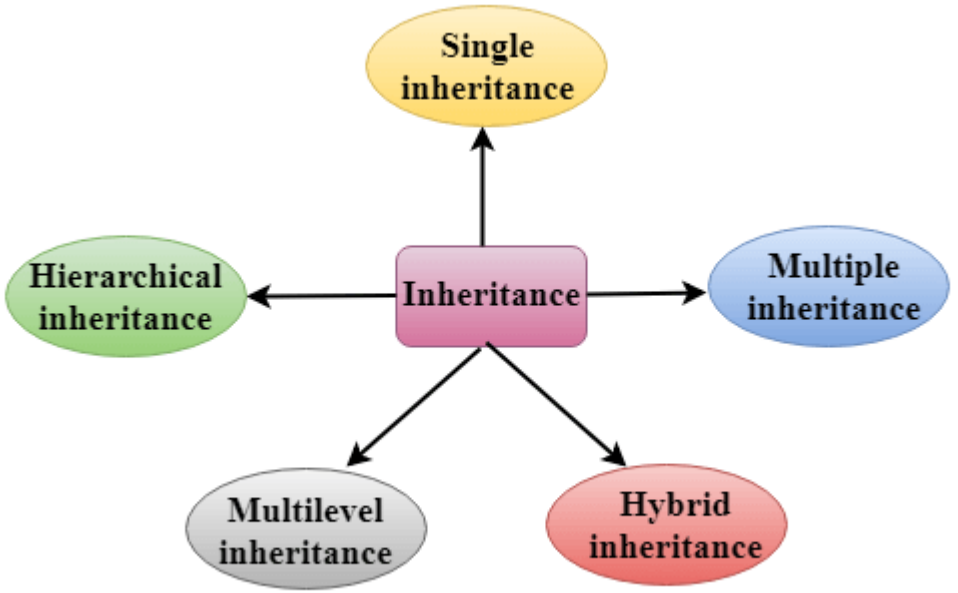
Single inheritance

Multiple inheritance

Hierarchical inheritance

Multilevel inheritance

Hybrid inheritance



Derived class

A Derived class is defined as the class derived from the base class.

The Syntax of Derived class:

```
class derived_class_name :: visibility-mode base_class_name
{
    // body of the derived class.
}
```

Where,

derived_class_name: It is the name of the derived class.

visibility mode: The visibility mode specifies whether the features of the base class are publicly inherited or privately inherited. It can be public or private.

base_class_name: It is the name of the base class.

When the base class is privately inherited by the derived class, public members of the base class becomes the private members of the derived class. Therefore, the public members of the base class are not accessible by the objects of the derived class only by the member functions of the derived class.

When the base class is publicly inherited by the derived class, public members of the base class also become the public members of the derived class. Therefore, the public members of the base class are accessible by the objects of the derived class as well as by the member functions of the base class.

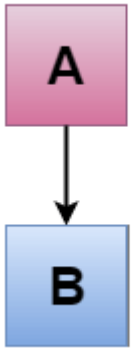
Note:

In C++, the default mode of visibility is private.

The private members of the base class are never inherited.

C++ Single Inheritance

Single inheritance is defined as the inheritance in which a derived class is inherited from the only one base class. Where 'A' is the base class, and 'B' is the derived class.



C++ Single Level Inheritance Example: Inheriting Fields

When one class inherits another class, it is known as single level inheritance.

Let's see the example of single level inheritance which inherits the fields only.

```
#include <iostream>
using namespace std;
 class Employee {
   public:
   float salary = 60000;
 };
   class Programmer: public  Employee  {
   public:
   float bonus = 5000;
   };
int main(void) {
     Programmer p1;
     cout<<"Salary: "<<p1.salary<<endl;
     cout<<"Bonus: "<<p1.bonus<<endl;
    return 0;
}
```

Output:
Salary: 60000
Bonus: 5000

In the above example, Employee is the **base** class and Programmer is the **derived** class.

C++ Single Level Inheritance Example: Inheriting Methods

```
#include <iostream>

using namespace std;

 class Animal {

   public:

 void eat() {

    cout<<"Eating..."<<endl;

 }

   };
```

```
 class Dog: public Animal
   {
      public:
     void bark(){
    cout<<"Barking...";
     }
   };
int main(void) {
    Dog d1;
    d1.eat();
    d1.bark();
    return 0;
}
```

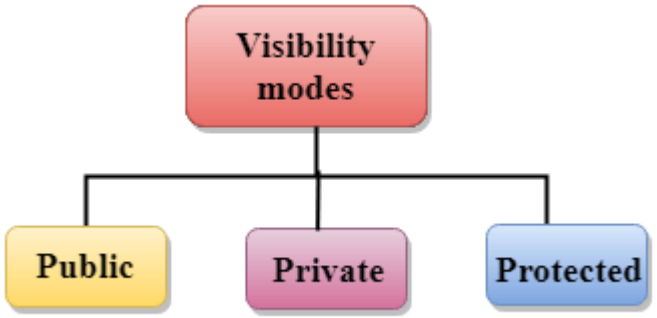
Output:
Eating... Barking...

How to make a Private Member Inheritable

The private member is not inheritable. If we modify the visibility mode by making it public, but this takes away the advantage of data hiding.

C++ introduces a third visibility modifier, i.e., **protected**. The member which is declared as protected will be accessible to all the member functions within the class as well as the class immediately derived from it.

Visibility modes can be classified into three categories:



Public: When the member is declared as public, it is accessible to all the functions of the program.

Private: When the member is declared as private, it is accessible within the class only.

Protected: When the member is declared as protected, it is accessible within its own class as well as the class immediately derived from it.

C++ Multilevel Inheritance

Multilevel inheritance is a process of deriving a class from another derived class.



C++ Multi Level Inheritance Example

When one class inherits another class which is further inherited by another class, it is known as multi level inheritance in C++. Inheritance is transitive so the last derived class acquires all the members of all its base classes.

Let's see the example of multi level inheritance in C++.

```
#include <iostream>
using namespace std;
 class Animal {
   public:
 void eat() {
    cout<<"Eating..."<<endl;


 }
   };
```

```
class Dog: public Animal
 {
    public:
  void bark(){
 cout<<"Barking..."<<endl;

  }
 };
```

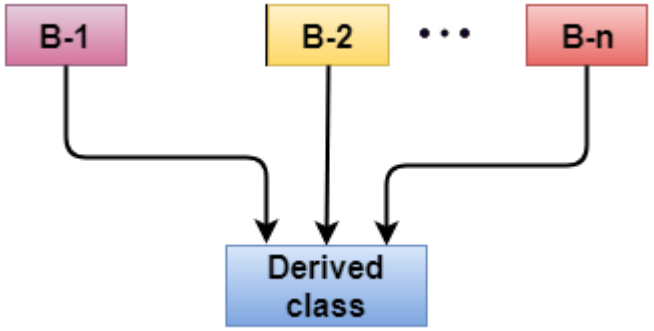
```
class BabyDog: public Dog
  {
     public:
    void weep() {
   cout<<"Weeping...";
    }
  };
 int main(void) {
    BabyDog d1;
    d1.eat();
    d1.bark();
     d1.weep();
     return 0;
}
```

Output:

```
Eating...
Barking...
Weeping...
```

C++ Multiple Inheritance

Multiple inheritance is the process of deriving a new class that inherits the attributes from two or more classes.



Syntax of the Derived class:

```
class D : visibility B-1, visibility B-2, ?
{
   // Body of the class;
}
```

```
#include <iostream>
using namespace std;
class A
{
   protected:
    int a;
   public:
   void get_a(int n)
   {
      a = n;
   }
};
```

```
class B
{
   protected:
   int b;
   public:
   void get_b(int n)
   {
      b = n;
   }
};
```

```
class C : public A,public B
{
   public:
    void display()
    {
       std::cout << "The value of a is : " <<a<< std::endl;
       std::cout << "The value of b is : " <<b<< std::endl;
       cout<<"Addition of a and b is : "<<a+b;
    }
};
```

```
 int main()
 {
    C c;
    c.get_a(10);
    c.get_b(20);
    c.display();

     return 0;
 1.}
```

Output:
The value of a is : 10
The value of b is : 20
Addition of a and b is : 30

Ambiguity Resolution in Inheritance

Ambiguity can be occurred in using the multiple inheritance when a function with the same name occurs in more than one base class.

Let's understand this through an example:

```
#include <iostream>
using namespace std;
class A
{
   public:
   void display()
   {
      std::cout << "Class A" << std::endl;
   }
};
```

```
class B
{
   public:
   void display()
   {
      std::cout << "Class B" << std::en
dl;
   }
};
```

```
class C : public A, public B
{
   void view()
   {
      display();
   }
};
int main()
{
   C c;
   c.display();
   return 0;
}
```

Output:
error: reference to 'display' is ambiguous display();

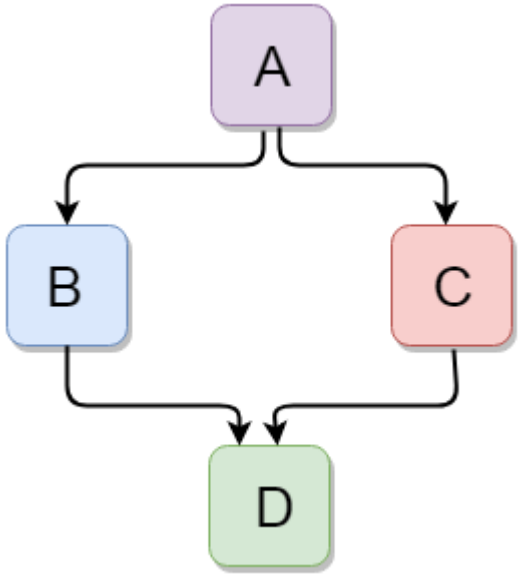
The above issue can be resolved by using the class resolution operator with the function. In the above example, the derived class code can be rewritten as:

```
class C : public A, public B
{
    void view()
    {
        A :: display();        // Calling the display() function of class A.
        B :: display();        // Calling the display() function of class B.

    }
};
```

C++ Hybrid Inheritence

Hybrid inheritance is a combination of more than one type of inheritance.



Example

```
#include <iostream>
using namespace std;
class A
{
   protected:
   int a;
   public:
   void get_a()
   {
     std::cout << "Enter the value of 'a' : " << std::endl;

     cin>>a;
   }
};
```

```
class B : public A
{
   protected:
   int b;
   public:
   void get_b()
   {
      std::cout << "Enter the value of 'b' : " << std::endl;

     cin>>b;
   }
};
```

```
class C
{
   protected:
   int c;
   public:
   void get_c()
   {
      std::cout << "Enter the value of c is : " << std::
endl;
      cin>>c;
   }
};
```

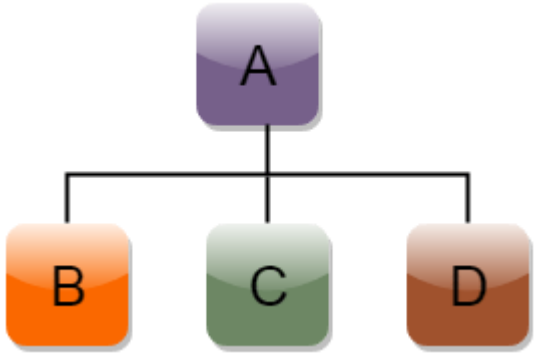
```
class D : public B, public C
{
   protected:
   int d;
   public:
   void mul()
   {
      get_a();
      get_b();
      get_c();
      std::cout << "Multiplication of a,b,c is : " <<a*b*c<
< std::endl;
   }
};
```

```
int main()
{
   D d;
   d.mul();
   return 0;
}
```

Output:
Enter the value of 'a' : 10 Enter the value of 'b' : 20 Enter the value of c is : 30 Multiplication of a,b,c is : 6000

C++ Hierarchical Inheritance

Hierarchical inheritance is defined as the process of deriving more than one class from a base class.



Syntax of Hierarchical inheritance:

```
class A
{
   // body of the class A.
}
class B : public A
{
   // body of class B.
}
class C : public A
{
   // body of class C.
}
class D : public A
{
   // body of class D.
}
```

```
#include <iostream>
using namespace std;
class Shape              // Declaration of base class.
{
   public:
   int a;
   int b;
   void get_data(int n,int m)
   {
      a= n;
      b = m;
   }
};

class Rectangle : public Shape  // inheriting Shape clas
s
{
   public:
   int rect_area()
   {
      int result = a*b;
      return result;
   }
};
```

```
class Triangle : public Shape    // inheriting Shape class

{
   public:
   int triangle_area()
   {
      float result = 0.5*a*b;
      return result;
   }
};
```

```
int main()
{
    Rectangle r;
    Triangle t;
    int length,breadth,base,height;
    std::cout << "Enter the length and breadth of a rectangle: " << std::endl;
    cin>>length>>breadth;
    r.get_data(length,breadth);
    int m = r.rect_area();
    std::cout << "Area of the rectangle is : " <<m<< std::endl;
    std::cout << "Enter the base and height of the triangle: " << std::endl;
    cin>>base>>height;
    t.get_data(base,height);
    float n = t.triangle_area();
    std::cout <<"Area of the triangle is : "  << n<<std::endl;
    return 0;
}
```

Output:

```
Enter the length and breadth of a rectangle:
23
20
 Area of the rectangle is : 460
Enter the base and height of the triangle:
2
5
Area of the triangle is : 5
```

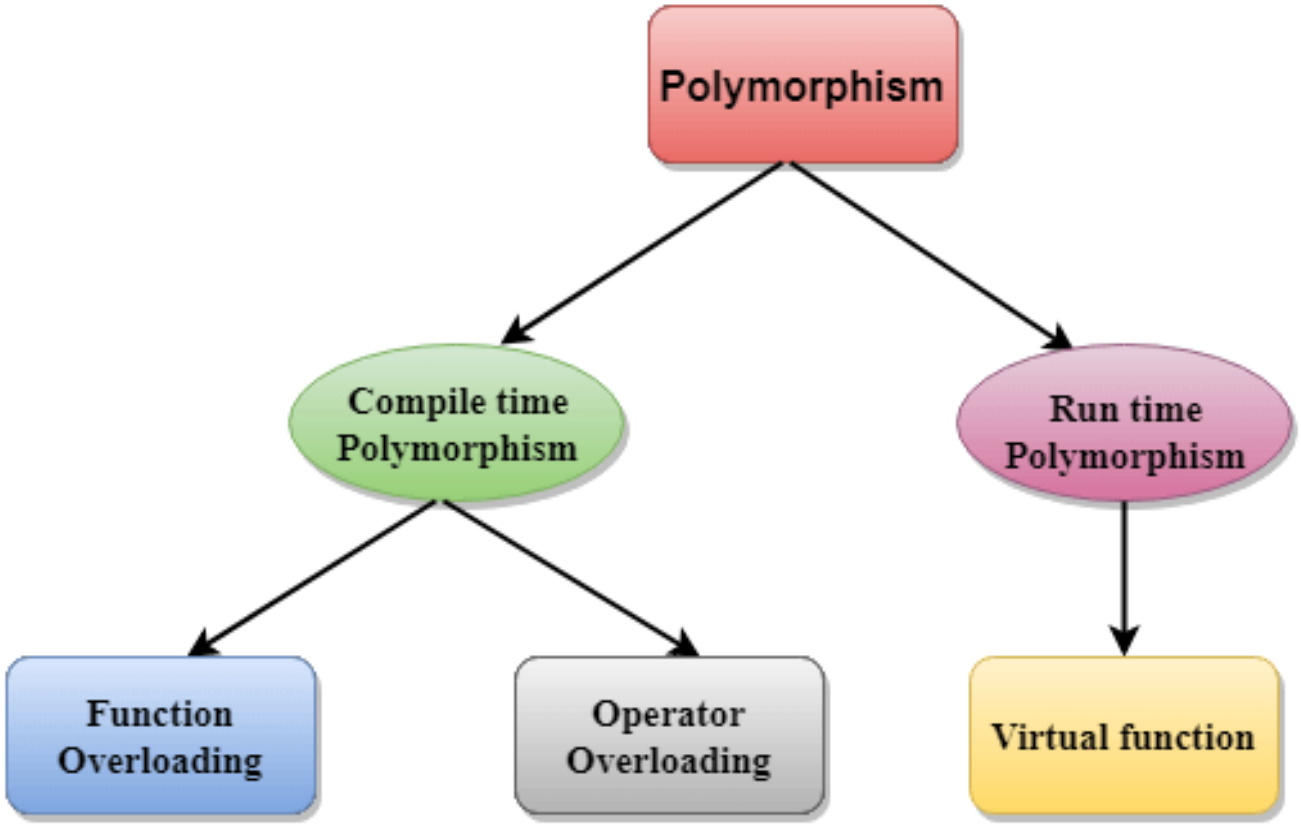
C++ Polymorphism

The term "Polymorphism" is the combination of "poly" + "morphs" which means many forms. It is a greek word. In object-oriented programming, we use 3 main concepts: inheritance, encapsulation, and polymorphism.

Real Life Example Of Polymorphism

Let's consider a real-life example of polymorphism. A lady behaves like a teacher in a classroom, mother or daughter in a home and customer in a market. Here, a single person is behaving differently according to the situations.

There are two types of polymorphism in C++:



Compile time polymorphism:

The overloaded functions are invoked by matching the type and number of arguments. This information is available at the compile time and, therefore, compiler selects the appropriate function at the compile time. It is achieved by function overloading and operator overloading which is also known as static binding or early binding. Now, let's consider the case where function name and prototype is same.

```
class A                          //  base class declaration.
 {
     int a;
     public:
     void display()
     {
          cout<< "Class A ";
      }
 };
class B : public A                    //  derived class declaration.
{
    int b;
    public:
   void display()
  {
       cout<<"Class B";
  }
};
```

In the above case, the prototype of display() function is the same in both the **base and derived class**. Therefore, the static binding cannot be applied. It would be great if the appropriate function is selected at the run time. This is known as **run time polymorphism**.

Run time polymorphism

Run time polymorphism is achieved when the object's method is invoked at the run time instead of compile time. It is achieved by method overriding which is also known as dynamic binding or late binding.

Differences b/w compile time and run time polymorphism.

Compile time polymorphism	Run time polymorphism
The function to be invoked is known at the compile time.	The function to be invoked is known at the run time.
It is also known as overloading, early binding and static binding.	It is also known as overriding, Dynamic binding and late binding.
Overloading is a compile time polymorphism where more than one method is having the same name but with the different number of parameters or the type of the parameters.	Overriding is a run time polymorphism where more than one method is having the same name, number of parameters and the type of the parameters.
It is achieved by function overloading and operator overloading.	It is achieved by virtual functions and pointers.
It provides fast execution as it is known at the compile time.	It provides slow execution as it is known at the run time.
It is less flexible as mainly all the things execute at the compile time.	It is more flexible as all the things execute at the run time.

C++ Runtime Polymorphism Example

Let's see a simple example of run time polymorphism in C++.
// an example without the virtual keyword.

```
#include <iostream>
using namespace std;
class Animal {
   public:
void eat(){
cout<<"Eating...";
   }
};
```

```
class Dog: public Animal
{
 public:
 void eat()  {
        cout<<"Eating bread...";
   }
};
int main(void) {
   Dog d = Dog();
   d.eat();
   return 0;
```

Output:
Eating bread...

C++ Run time Polymorphism Example: By using two derived class

Let's see another example of run time polymorphism in C++ where we are having two derived classes.

// an example with virtual keyword.

```
#include <iostream>
using namespace std;
class Shape {                    //  base class
   public:
virtual void draw()
{            // virtual function
cout<<"drawing..."<<endl;
   }
};
```

```
class Rectangle: public Shape //  inher
iting Shape class.
{
 public:
 void draw()
   {
      cout<<"drawing rectangle..."<<end
l;
    }
};
```

```
class Circle: public Shape
 //  inheriting Shape class.

{
 public:
 void draw()
   {
     cout<<"drawing circle..."<<endl;
   }
};
```

```
int main(void) {
    Shape *s;                    //  base class pointer.
    Shape sh;                    // base class object.
    Rectangle rec;
    Circle cir;
      s=&sh;
     s->draw();
        s=&rec;
     s->draw();
    s=&cir
    s->draw();
}
```

Output:

drawing...
drawing rectangle...
drawing circle...

Runtime Polymorphism with Data Members

Runtime Polymorphism can be achieved by data members in C++. Let's see an example where we are accessing the field by reference variable which refers to the instance of derived class.

```
#include <iostream>
using namespace std;
class Animal {       //  base class declara
tion.
    public:
    string color = "Black";
};
```

```
class Dog: public Animal       // inheriting Anim
al class.
{
 public:
    string color = "Grey";
};
int main(void) {
     Animal d= Dog();
    cout<<d.color;
}
```

**Outpu
t:**
Black

Overloading

If we create two or more members having the same name but different in number or type of parameter, it is known as C++ overloading. In C++, we can overload:

methods,

constructors, and

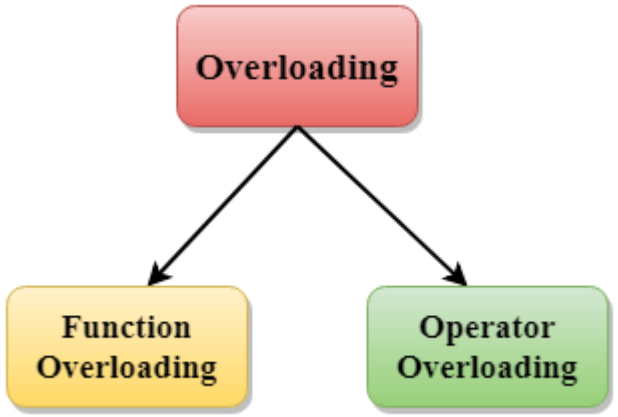
indexed properties

It is because these members have parameters only.

Types of overloading in C++ are:

Function overloading

Operator overloading



C++ Function Overloading

Function Overloading is defined as the process of having two or more function with the same name, but different in parameters is known as function overloading in C++. In function overloading, the function is redefined by using either different types of arguments or a different number of arguments. It is only through these differences compiler can differentiate between the functions.

The advantage of Function overloading is that it increases the readability of the program because you don't need to use different names for the same action.

C++ Function Overloading Example

Let's see the simple example of function overloading where we are changing number of arguments of add() method.

// program of function overloading when number of arguments vary.

```
#include <iostream>
using namespace std;
class Cal {
    public:
static int add(int a,int b){
        return a + b;
    }
static int add(int a, int b, int c)
    {
        return a + b + c;
    }
};
```

```
int main(void) {
    Cal C;
     //     class object declaration.
    cout<<C.add(10, 20)<<endl;
    cout<<C.add(12, 20, 23);
   return 0;
}
```

Output:

```
30
55
```

Let's see the simple example when the type of the arguments vary.

// Program of function overloading with different types of arguments.

```
#include<iostream>
using namespace std;
int mul(int,int);
float mul(float,int);


int mul(int a,int b)
{
   return a*b;
}
float mul(double x, int y)
{
   return x*y;
}
```

```
int main()
{
   int r1 = mul(6,7);
   float r2 = mul(0.2,3);
   std::cout << "r1 is : " <<r1<< std::endl;
   std::cout <<"r2 is : "  <<r2<< std::endl;
   return 0;
}
```

Output:

```
r1 is : 42
r2 is : 0.6
```

Function Overloading and Ambiguity

When the compiler is unable to decide which function is to be invoked among the overloaded function, this situation is known as **function overloading**.

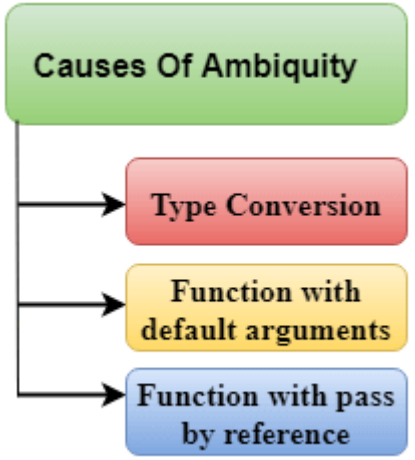
When the compiler shows the ambiguity error, the compiler does not run the program.

Causes of Function Overloading:

Type Conversion.

Function with default arguments.

Function with pass by reference.



Type Conversion

```
#include<iostream>
using namespace std;
void fun(int);
void fun(float);
void fun(int i)
{
   std::cout << "Value of i is : " <<i<< std::endl;
}
void fun(float j)
{
   std::cout << "Value of j is : " <<j<< std::endl;
}
```

```
int main()
{
   fun(12);
   fun(1.2);
   return 0;
}
```

The above example shows an error "**call of overloaded 'fun(double)' is ambiguous**". The fun(10) will call the first function. The fun(1.2) calls the second function according to our prediction. But, this does not refer to any function as in C++, all the floating point constants are treated as double not as a float. If we replace float to double, the program works. Therefore, this is a type conversion from float to double.

Function with Default Arguments

```
#include<iostream>
using namespace std;
void fun(int);
void fun(int,int);
void fun(int i)
{
   std::cout << "Value of i is : " <<i<< std::endl;
}
void fun(int a,int b=9)
{
   std::cout << "Value of a is : " <<a<< std::endl;
   std::cout << "Value of b is : " <<b<< std::endl;
}
int main()
{
   fun(12);

   return 0;
}
```

The above example shows an error "call of overloaded 'fun(int)' is ambiguous". The fun(int a, int b=9) can be called in two ways: first is by calling the function with one argument, i.e., fun(12) and another way is calling the function with two arguments, i.e., fun(4,5). The fun(int i) function is invoked with one argument. Therefore, the compiler could not be able to select among fun(int i) and fun(int a,int b=9).

•Function with pass by reference

```
#include <iostream>
using namespace std;
void fun(int);
void fun(int &);
int main()
{
int a=10;
fun(a); // error, which f()?
return 0;
}
```

```
void fun(int x)
{
std::cout << "Value of x is : " <<x<< std::endl;
}
void fun(int &b)
{
std::cout << "Value of b is : " <<b<< std::endl;
}
```

The above example shows an error "**call of overloaded 'fun(int&)' is ambiguous**". The first function takes one integer argument and the second function takes a reference parameter as an argument. In this case, the compiler does not know which function is needed by the user as there is no syntactical difference between the fun(int) and fun(int &).

C++ Operators Overloading

Operator overloading is a compile-time polymorphism in which the operator is overloaded to provide the special meaning to the user-defined data type. Operator overloading is used to overload or redefines most of the operators available in C++. It is used to perform the operation on the user-defined data type. For example, C++ provides the ability to add the variables of the user-defined data type that is applied to the built-in data types.

The advantage of Operators overloading is to perform different operations on the same operand.

Operator that cannot be overloaded are as follows:

Scope operator (::)

Sizeof

member selector(.)

member pointer selector(*)

ternary operator(?:)

Syntax of Operator Overloading

```
return_type class_name  : : operator op(argument_list)
{
    // body of the function.
}
```

Where the **return type** is the type of value returned by the function.

class_name is the name of the class.

operator op is an operator function where op is the operator being overloaded, and the operator is the keyword.

Rules for Operator Overloading

Existing operators can only be overloaded, but the new operators cannot be overloaded.

The overloaded operator contains atleast one operand of the user-defined data type.

We cannot use friend function to overload certain operators. However, the member function can be used to overload those operators.

When unary operators are overloaded through a member function take no explicit arguments, but, if they are overloaded by a friend function, takes one argument.

When binary operators are overloaded through a member function takes one explicit argument, and if they are overloaded through a friend function takes two explicit arguments.

```
#include <iostream>
using namespace std;
class Test
{
   private:
      int num;
   public:
      Test(): num(8){}
      void operator ++()       {
         num = num+2;
      }
      void Print() {
          cout<<"The Count is: "<<num;

      }
};
```

```
int main()
{
    Test tt;
    ++tt;  // calling of a function "void operator ++()"
    tt.Print();
    return 0;
}
```

Let's see a simple example of overloading the binary operators.

// program to overload the binary operators.

```
#include <iostream>
using namespace std;
class A
{

   int x;
     public:
     A(){}
   A(int i)
   {
      x=i;
   }
   void operator+(A);
   void display();
};
void A :: operator+(A a)
{

   int m = x+a.x;
   cout<<"The result of the addition of two objects is : "
<<m;

}
int main()
{
   A a1(5);
   A a2(4);
   a1+a2;
   return 0;
}
```

Function Overriding

If derived class defines same function as defined in its base class, it is known as function overriding in C++. It is used to achieve runtime polymorphism. It enables you to provide specific implementation of the function which is already provided by its base class.

C++ Function Overriding Example

Let's see a simple example of Function overriding in C++. In this example, we are overriding the eat() function.

```
#include <iostream>
using namespace std;
class Animal {
   public:
void eat(){
cout<<"Eating...";
   }
};
class Dog: public Animal
{
 public:
 void eat()
   {
     cout<<"Eating bread...";
   }
};
int main(void) {
   Dog d = Dog();
   d.eat();
   return 0;
}
```

Output:

Eating bread...

C++ virtual function

A C++ virtual function is a member function in the base class that you redefine in a derived class. It is declared using the virtual keyword.

It is used to tell the compiler to perform dynamic linkage or late binding on the function.

There is a necessity to use the single pointer to refer to all the objects of the different classes. So, we create the pointer to the base class that refers to all the derived objects. But, when base class pointer contains the address of the derived class object, always executes the base class function. This issue can only be resolved by using the 'virtual' function.

A 'virtual' is a keyword preceding the normal declaration of a function.

When the function is made virtual, C++ determines which function is to be invoked at the runtime based on the type of the object pointed by the base class pointer.

Late binding or Dynamic linkage

In late binding function call is resolved during runtime. Therefore compiler determines the type of object at runtime, and then binds the function call.

Rules of Virtual Function

Virtual functions must be members of some class.

Virtual functions cannot be static members.

They are accessed through object pointers.

They can be a friend of another class.

A virtual function must be defined in the base class, even though it is not used.

The prototypes of a virtual function of the base class and all the derived classes must be identical. If the two functions with the same name but different prototypes, C++ will consider them as the overloaded functions.

We cannot have a virtual constructor, but we can have a virtual destructor

Consider the situation when we don't use the virtual keyword.

```
#include <iostream>
using namespace std;
class A
{
   int x=5;
    public:
    void display()
    {
        std::cout << "Value of x is : " << x<<std::endl;
    }
};
class B: public A
{
    int y = 10;
    public:
    void display()
    {
        std::cout << "Value of y is : " <<y<< std::endl;
    }
};
int main()
{
    A *a;
    B b;
    a = &b;
   a->display();
    return 0;
}
```

Output:
Value of x is : 5

In the above example, * a is the base class pointer. The pointer can only access the base class members but not the members of the derived class. Although C++ permits the base pointer to point to any object derived from the base class, it cannot directly access the members of the derived class. Therefore, there is a need for virtual function which allows the base pointer to access the members of the derived class.

C++ virtual function Example

Let's see the simple example of C++ virtual function used to invoked the derived class in a program

```
#include <iostream>
{
class A{
 public:
 virtual void display()
 {
  cout << "Base class is invoked"<<endl;
 }
};
class B:public A
{
 public:
 void display()
 {
  cout << "Derived Class is invoked"<<endl;
 }
};
int main()
{
 A* a;    //pointer of base class
 B b;     //object of derived class
 a = &b;
 a->display();   //Late Binding occurs
}
```

Output:

Derived Class is invoked

Pure Virtual Function

A virtual function is not used for performing any task. It only serves as a placeholder.

When the function has no definition, such function is known as "**do-nothing**" function.

The "**do-nothing**" function is known as a **pure virtual function**. A pure virtual function is a function declared in the base class that has no definition relative to the base class.

A class containing the pure virtual function cannot be used to declare the objects of its own, such classes are known as abstract base classes.

The main objective of the base class is to provide the traits to the derived classes and to create the base pointer used for achieving the runtime polymorphism.

Pure virtual function can be defined as:

virtual void display() = 0;

```
#include <iostream>
using namespace std;
class Base
{
   public:
   virtual void show() = 0;
};
class Derived : public Base
{
   public:
   void show()
   {
      std::cout << "Derived class is derived from the bas
e class." << std::endl;
   }
1.};
```

```
int main()
{
   Base *bptr;
   //Base b;
   Derived d;
   bptr = &d;
   bptr->show();
   return 0;
}
```

Output:

Derived class is derived from the base class.

In the above example, the base class contains the pure virtual function. Therefore, the base class is an abstract base class. We cannot create the object of the base class.

Abstract Classes

Interfaces in C++ (Abstract Classes)

Abstract classes are the way to achieve abstraction in C++. Abstraction in C++ is the process to hide the internal details and showing functionality only. Abstraction can be achieved by two ways:

Abstract class

Interface

Abstract class and interface both can have abstract methods which are necessary for abstraction.

C++ Abstract class

In C++ class is made abstract by declaring at least one of its functions as <>strong>pure virtual function. A pure virtual function is specified by placing "= 0" in its declaration. Its implementation must be provided by derived classes.

Let's see an example of abstract class in C++ which has one abstract method draw(). Its implementation is provided by derived classes: Rectangle and Circle. Both classes have different implementation.

```
#include <iostream>
using namespace std;
 class Shape
{
   public:
   virtual void draw()=0;
};
 class Rectangle : Shape
{
   public:
    void draw()
   {
      cout < <"drawing rectangle..." < <endl;
   }
};
```

```
class Circle : Shape
{
   public:
    void draw()
   {
      cout <<"drawing circle..." < <endl;
   }
};
int main( ) {
   Rectangle rec;
   Circle cir;
   rec.draw();
   cir.draw();
  return 0;
}
```

Output:
Drawing rectangle...
drawing circle...

Data Abstraction in C++

Data Abstraction is a process of providing only the essential details to the outside world and hiding the internal details, i.e., representing only the essential details in the program.

Data Abstraction is a programming technique that depends on the seperation of the interface and implementation details of the program.

Let's take a real life example of AC, which can be turned ON or OFF, change the temperature, change the mode, and other external components such as fan, swing. But, we don't know the internal details of the AC, i.e., how it works internally. Thus, we can say that AC seperates the implementation details from the external interface.

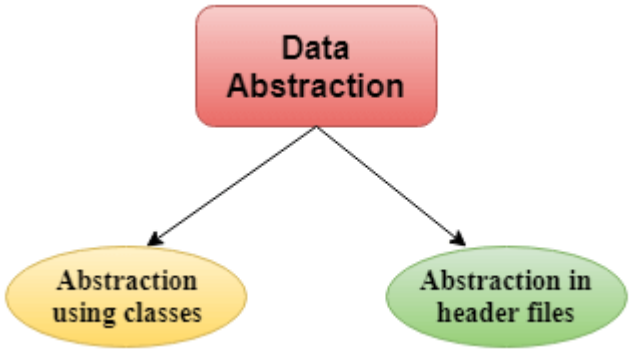
C++ provides a great level of abstraction. For example, pow() function is used to calculate the power of a number without knowing the algorithm the function follows.

In C++ program if we implement class with private and public members then it is an example of data abstraction.

Data Abstraction can be achieved in two ways:

Abstraction using classes

Abstraction in header files.



Abstraction using classes:

An abstraction can be achieved using classes. A class is used to group all the data members and member functions into a single unit by using the access specifiers. A class has the responsibility to determine which data member is to be visible outside and which is not.

Abstraction in header files:

An another type of abstraction is header file. For example, pow() function available is used to calculate the power of a number without actually knowing which algorithm function uses to calculate the power. Thus, we can say that header files hides all the implementation details from the user.

Access Specifiers Implement Abstraction:

Public specifier: When the members are declared as public, members can be accessed anywhere from the program.

Private specifier: When the members are declared as private, members can only be accessed only by the member functions of the class.

Let's see a simple example of abstraction in header files.

```
#include <iostream>
#include<math.h>
using namespace std;
int main()
{
 int n = 4;
   int power = 3;
   int result = pow(n,power);        // pow(n,power) is the  power function
   std::cout << "Cube of n is : " <<result<< std::endl;
   return 0;
}
```

In the above example, pow() function is used to calculate 4 raised to the power 3. The pow() function is present in the math.h header file in which all the implementation details of the pow() function is hidden.

```
#include <iostream>
using namespace std;
 class Sum
{
private: int x, y, z; // private variables
public:
void add()
{
cout<<"Enter two numbers: ";
cin>>x>>y;
z= x+y;
cout<<"Sum of two number is: "<<z<<endl;
}
};
int main()
{
Sum sm;
sm.add();
return 0;
}
```

Output:

Enter two numbers:

3

6

Sum of two number is: 9

In the above example, abstraction is achieved using classes. A class 'Sum' contains the private members x, y and z are only accessible by the member functions of the class.

Advantages Of Abstraction:

Implementation details of the class are protected from the inadvertent user level errors.

A programmer does not need to write the low level code.

Data Abstraction avoids the code duplication, i.e., programmer does not have to undergo the same tasks every time to perform the similar operation.

The main aim of the data abstraction is to reuse the code and the proper partitioning of the code across the classes.

Internal implementation can be changed without affecting the user level code.

Organizing C++ Classes Across Multiple Files: Best Practices for Header and Source Files

In C++, it's common practice to split classes into multiple files to improve code organization, readability, and maintainability. Typically, a class is divided into a header file (with a .h or .hpp extension) and a source file (with a .cpp extension).

Steps to define and use C++ classes in different files

1. Header File (ClassName.h)

The header file contains the class declaration, which includes the class definition, member variables, and member function prototypes (but no actual implementation of the functions).

MyClass.h

```
#ifndef MYCLASS_H
#define MYCLASS_H

class MyClass {
public:
    MyClass();          // Constructor
    void display();     // Member function

private:
    int value;          // Private member variable
};

#endif
```

Include Guards (#ifndef, #define, #endif) are used to prevent multiple inclusions of the same header file.The class declaration in the header file defines what the class will look like without providing the actual logic.

Include Guards (#ifndef, #define, #endif) are used to prevent multiple inclusions of the same header file.

The class declaration in the header file defines what the class will look like without providing the actual logic.

2. Source File (ClassName.cpp)

The source file contains the actual implementation of the class's member functions. You include the corresponding header file here so that the compiler knows the class structure.

```
#include "MyClass.h"
#include <iostream>

MyClass::MyClass() : value(0) {
    // Constructor implementation
}

void MyClass::display() {
    std::cout << "Value: " << value << std::endl;
}
```

Include the header file (#include "MyClass.h") to inform the compiler about the class declaration.Each function is prefixed with ClassName:: to indicate that the function belongs to the MyClass class.

3. Main Program File (main.cpp)

```
#include "MyClass.h"


int main() {
    MyClass obj;    // Create object of MyClass
    obj.display();  // Call the display function
    return 0;
}
```

The main program only needs to include the header file, not the .cpp file, because the implementation details are taken care of during the linking process after compilation.

Compilation

If you're compiling these files manually, you can use a command like the following:

```
g++ main.cpp MyClass.cpp -o program
```

This compiles both the main program and the class implementation and links them into a single executable named program.

Summary of File Structure

MyClass.h: Contains the class declaration (interface).
MyClass.cpp: Contains the implementation of the class methods.
main.cpp: The program that uses the class.

Example 2

Adder.h

```
#ifndef ADDER_H
#define ADDER_H


class Adder {
public:
    Adder();                // Default constructor
    int add(int a, int b);  // Member function to add two numbers
};


#endif
```

Adder.cpp

```
#include "Adder.h"


Adder::Adder() {
    // Constructor (can initialize things if needed)
}


int Adder::add(int a, int b) {
    return a + b;   // Perform addition and return result
}
```

Main2.cpp

```
#include <iostream>
#include "Adder.h"
using namespace std;

int main() {
    Adder calc;           // Create an object of Adder
    int num1, num2;

    cout << "Enter first number: ";
    cin >> num1;

    cout << "Enter second number: ";
    cin >> num2;

    int result = calc.add(num1, num2);  // Call the add() function

    cout << "The sum is: " << result << endl;
    return 0;
}
```

Compile and RUN

g++ main2.cpp Adder.cpp -o program2

./program2

Benefits of This Structure

Separation of Concerns: The header file describes the "what" (interface), while the source file describes the "how" (implementation).

Reusability: Header files can be reused in multiple projects or modules without copying code.Reduced

Compilation Time: By splitting files, changes to the implementation file don’t require recompiling files that only use the header file.